

FORMATION EMBARQUÉE

TP2 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Fiche de séance 3 : robustesse et module d'écho (3 h)

En-tête pédagogique

Objectifs	Tester aux limites (glitch, horloges désaccordées) ; assembler un système hiérarchique ; gérer un conflit de ressource ; (option) passer sur carte réelle
Prérequis	Séances 1-2 : boucle 256/256 verte
Outils	GHDL + GTKWave ; (option) Vivado/Quartus + carte
Livrable	3 tests de robustesse passés + module d'écho fonctionnel

Déroulé minuté

0:00-0:45 — Test 1 : le glitch anti-parasite

Ajoute au testbench un stimulus qui force un faux start hors trame :

```
-- entre deux octets, un parasite plus court qu'un demi-bit
fil <= '0'; wait for 3 us;    -- 3 µs < demi-bit (4,34 µs) : doit être ignoré
fil <= '1'; wait for 20 us;
```

Attendu : `valide` ne se lève JAMAIS pour ce parasite. C'est la vérification à mi-start (séance 2) qui fait le travail. Observe au chronogramme : la FSM va en VERIF_START puis **retourne à REPOS** car `sync1` est remonté à 1 au milieu.

0:45-1:45 — Test 2 : horloges désaccordées (le vrai test réel)

Dans la vraie vie, l'émetteur et le récepteur ont des quartz légèrement différents. Instancie-les avec des `F_CLK` distincts :

```
emetteur : entity work.uart_tx
    generic map (F_CLK => 50_000_000, BAUD => 115_200) port map (...);
recepteur : entity work.uart_rx
    generic map (F_CLK => 49_000_000, BAUD => 115_200) port map (...);
-- -2 % : le récepteur croit compter 50 MHz mais l'horloge fait 49
```

À **-2 %** : la boucle 256/256 doit encore passer (marge du milieu de bit). À **-5 %** : elle échoue — et c'est le but. Calcule pourquoi : erreur de 5 % par bit $\times$ $\sim 9,5$ bits (du milieu du start au milieu du bit 7) ≈ 47 % d'un bit $\rightarrow$ on frôle le bord, un octet finit par se décaler. **Documente ce calcul** : c'est la démonstration que le sur-échantillonnage a une limite quantifiable, pas magique.

Note pratique : ici les deux entités partagent la MÊME horloge de simulation ; le désaccord est *émulé* par le `F_CLK` que le récepteur croit avoir. Pour un désaccord physique réel, on générerait deux

1:45-2:30 — Module d'écho hiérarchique

Assemble un `top_echo` qui renvoie chaque octet reçu :

```
architecture rtl of top_echo is
    signal rx_data : std_logic_vector(7 downto 0);
    signal rx_valide, tx_busy, tx_start : std_logic;
    signal registre_attente : std_logic_vector(7 downto 0);
    signal octet_en_attente : std_logic := '0';
begin
    u_rx : entity work.uart_rx port map (clk=>clk, rx=>rx,
                                         donnee=>rx_data, valide=>rx_valide);
    u_tx : entity work.uart_tx port map (clk=>clk, tx_start=>tx_start,
                                         tx_data=>registre_attente,
                                         tx=>tx, busy=>tx_busy);

    -- Gestion du cas "TX occupé quand un octet arrive" : un registre d'attente
    process (clk)
    begin
        if rising_edge(clk) then
            tx_start <= '0'; -- impulsion par défaut
            if rx_valide = '1' then
                registre_attente <= rx_data; -- mémoriser
                octet_en_attente <= '1';
            end if;
            if octet_en_attente = '1' and tx_busy = '0' then
                tx_start <= '1'; -- émettre dès que TX libre
                octet_en_attente <= '0';
            end if;
        end if;
    end process;
end architecture;
```

Décision de conception à documenter : ici on mémorise UN octet. Si un 2^e arrive avant que le 1^e soit émis, il est perdu. Pour n'en perdre aucun, il faudrait un **ring buffer** (celui du TD 01 !) entre RX et TX — mentionne cette limite dans ton compte rendu, c'est ce qu'un ingénieur écrit.

Testbench : envoie « VHDL » octet par octet, vérifie l'écho.

2:30-2:50 — (option) Passage sur carte réelle

Si tu as une Basys 3 (ou autre) :

```
# fichier de contraintes .xdc (Basys 3) – extrait
set_property PACKAGE_PIN W5 [get_ports clk]
create_clock -period 10.0 [get_ports clk]          # 100 MHz -> adapter le generic !
set_property PACKAGE_PIN B18 [get_ports rx]        # USB-UART RXD
set_property PACKAGE_PIN A18 [get_ports tx]        # USB-UART TXD
set_property IOSTANDARD LVCMOS33 [get_ports {clk rx tx}]
```

⚠ `generic map (F_CLK => 100_000_000)` pour la Basys 3 ! Synthétise, charge, ouvre un terminal série (`picocom -b 115200 /dev/ttyUSB1`) : ce que tu tapes revient à l'écran → ton matériel parle au PC.

2:50-3:00 — Livrables et barème

Vérifie ta note avec la grille du TP principal (§Livrables). Commit final :

```
git add . && git commit -m "TP2 seance 3 : robustesse + echo hierarchique"
```

Erreurs fréquentes

Symptôme	Cause	Remède
Le glitch génère un octet 0x00	vérif à mi-start absente	ajouter l'état VERIF_START
-2 % échoue déjà	échantillonnage pas au milieu (marge nulle)	revoir le demi-bit initial
Écho perd des octets en rafale	registre d'attente unique (attendu) ou pas de mémorisation	ring buffer si zéro perte requis
Sur carte : rien ne s'affiche	generic F_CLK pas adapté à l'horloge de la carte	100 MHz sur Basys 3
Sur carte : caractères corrompus	mauvais quartz déclaré, ou baudrate terminal ≠	vérifier <code>.xdc</code> et picocom

Bilan du TP 2

Tu as conçu, simulé et validé un périphérique matériel complet avec la méthode pro : spec chiffrée → schéma → code → testbench exhaustif → tests aux limites → intégration. C'est exactement le workflow d'un ingénieur FPGA. Le tout est réutilisable : `uart_tx/rx` te resserviront pour envoyer les mesures d'un projet FPGA vers un PC.

➡ Retour : [TP 2 \(vue d'ensemble\)](#) · [Module 05 — Java](#)